

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)
Assessment Tool Weightage: 50%

ROHIT L – 192511236

Annexure A: Constraint-Based Problem Solving

Solutions to CSP Doctor Shift Scheduling, Grid Pathfinding Search, and Autonomous Disaster Rescue Agent

1. CSP Doctor Shift Scheduling (Backtracking Search)

Problem Statement: Assign 3 doctors (D1, D2, D3) to 3 shifts (Morning [M], Afternoon [A], Night [N]) satisfying all specified scheduling constraints using Backtracking Search.

Formal Constraint Satisfaction Problem (CSP) Formulation

- **Variables (X):** D1, D2, D3 (the three doctors requiring shift assignments)
- **Domains (D):** For each doctor, $D_i = \{\text{Morning (1), Afternoon (2), Night (3)}\}$
- **Constraints (C):**
 - C1 (Shift Prohibition): $D1 \neq \text{Night}$ ($D1 \in \{\text{Morning, Afternoon}\}$)
 - C2 (Shift Prohibition): $D3 \neq \text{Morning}$ ($D3 \in \{\text{Afternoon, Night}\}$)
 - C3 (Order Constraint): $\text{Shift}(D2) < \text{Shift}(D3)$ [where Morning < Afternoon < Night]
 - C4 (All-Different / Exact Assignment): All doctors must have unique shifts: $\text{Shift}(D1) \neq \text{Shift}(D2)$, $\text{Shift}(D2) \neq \text{Shift}(D3)$, $\text{Shift}(D1) \neq \text{Shift}(D3)$

Initial Domain Reduction (Arc Consistency / Forward Checking)

Applying unary constraints directly before starting search:

- D1 domain: {Morning, Afternoon}
- D2 domain: {Morning, Afternoon, Night}
- D3 domain: {Afternoon, Night}

Step-by-Step Backtracking Execution Trace

Step	Variable Selected	Value Assignment	Constraint Checks & Consistency	Search Decision / Action
1	D1	Morning	Check C1 ($D1 \neq \text{Night}$): PASS.	Consistent. Proceed to D2.
2	D2	Morning	Check C4 (AllDiff): $\text{Shift}(D1)=\text{Shift}(D2)=\text{Morning}$. FAIL!	Inconsistent. Backtrack D2.

3	D2	Afternoon	Check C4: Shift(D2)≠Shift(D1). PASS.	Consistent. Proceed to D3.
4	D3	Afternoon	Check C4: Shift(D3)=Shift(D2)=Aft ernoon. FAIL!	Inconsistent. Try next value for D3.
5	D3	Night	Check C2 (D3≠M): PASS. Check C3 (D2<D3): Afternoon < Night. PASS. Check C4 (AllDiff): {M, A, N} unique. PASS.	SOLUTION FOUND! Terminate search.

Final Valid Schedule

Final Valid Assignment:

- D1 = Morning Shift
- D2 = Afternoon Shift
- D3 = Night Shift

Constraint Verification Check:

- ✓ D1 (Morning) is not Night
- ✓ D2 (Afternoon) works before D3 (Night)
- ✓ D3 (Night) is not Morning
- ✓ Exactly one doctor per shift with all 3 shifts covered

2. 5x5 Grid Environment Pathfinding (A* / Best-First Search)

Problem Setup: A robot moves in a 5x5 grid from Start S = (0,0) to Goal G = (4,4). Cost per move = 1. Obstacles block cells (1,1), (1,2), (2,3), and (3,1). Movements allowed: Up, Down, Left, Right.

Manhattan Distance Heuristic Definition

The Manhattan distance heuristic is defined as: $h(x, y) = |x - G_x| + |y - G_y| = |x - 4| + |y - 4|$

Heuristic Grid Values for 5x5 Grid

y \ x	Col 0	Col 1	Col 2	Col 3	Col 4
Row 0	h=8 (S)	h=7	h=6	h=5	h=4
Row 1	h=7	BLOCKED	BLOCKED	h=4	h=3
Row 2	h=6	h=5	h=4	BLOCKED	h=2
Row 3	h=5	BLOCKED	h=3	h=2	h=1
Row 4	h=4	h=3	h=2	h=1	h=0 (G)

Step-by-Step Node Expansion & Priority Queue Trace (f = g + h)

Step	Pop Node	g, h, f	Successors Generated & Evaluated	Priority Queue State [Node: f=g+h]
0	-	-	-	[(0,0): g=0, h=8, f=8]
1	(0,0)	g=0, h=8, f=8	(1,0): g=1, h=7, f=8 (0,1): g=1, h=7, f=8	[(1,0): f=8], [(0,1): f=8]
2	(1,0)	g=1, h=7, f=8	(2,0): g=2, h=6, f=8 (1,1) is BLOCKED	[(2,0): f=8], [(0,1): f=8]
3	(2,0)	g=2, h=6, f=8	(3,0): g=3, h=5, f=8 (2,1): g=3, h=5, f=8	[(3,0): f=8], [(2,1): f=8], [(0,1): f=8]
4	(3,0)	g=3, h=5, f=8	(4,0): g=4, h=4, f=8 (3,1) is BLOCKED	[(4,0): f=8], [(2,1): f=8], [(0,1): f=8]
5	(4,0)	g=4, h=4, f=8	(4,1): g=5, h=3, f=8	[(4,1): f=8], [(2,1): f=8], [(0,1): f=8]
6	(4,1)	g=5, h=3, f=8	(4,2): g=6, h=2, f=8	[(4,2): f=8], [(2,1): f=8], [(0,1): f=8]
7	(4,2)	g=6, h=2, f=8	(4,3): g=7, h=1, f=8	[(4,3): f=8], [(2,1): f=8], [(0,1): f=8]
8	(4,3)	g=7, h=1, f=8	(4,4): g=8, h=0, f=8 (GOAL)	[(4,4): f=8], [(2,1): f=8], [(0,1): f=8]
9	(4,4)	g=8, h=0, f=8	Goal Node Popped! Search Complete.	Search Terminated.

Optimal Path & Total Cost

Optimal Trajectory:

(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Optimal Path Cost: g(G) = 8 moves

3. Autonomous Disaster Rescue Robot Agent Analysis

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors under partial observability, dynamic hazards, and movement costs.

a) Analysis of Constraints & Impact on Robot Decision-Making

- **4-Directional Grid Movement:** Restricts spatial transitions to cardinal directions {Up, Down, Left, Right}, limiting branching factor $b \leq 4$.
- **Blocked Cells (Obstacles):** Structural collapses block paths unpredictably, forcing continuous local replanning upon collision/sensor detection.
- **Limited Sensing Capability (Partial Observability):** The robot cannot observe distant rooms or full building layout; it can only sense immediate adjacent cells. Decisions must be made under state uncertainty using a belief state model.
- **Variable Path Costs (Base=1, Risky Zone=2):** Simple hop-count (BFS) fails to guarantee optimal/safe navigation. The agent must evaluate total accumulated edge cost $g(n)$ to balance distance against hazard exposure.
- **Cost Minimization & Safe Navigation:** Creates a multi-objective utility trade-off where safety dominates speed, requiring risk-aware cost formulation.
- **Dynamic Knowledge Updating (Online Search Scenario):** The robot must combine exploration (discovering new cells) with exploitation (following lowest-cost known path) and update its map dynamically in real time.

b) Solution Approach & Search Strategy (Online LRTA* / D* Lite / Online UCS)

The optimal architecture is an Online Search Agent incorporating Learning Real-Time A* (LRTA*) or incremental search (D* Lite) with dynamic map updating.

Step-by-Step Execution Lifecycle

Phase	Agent Action / State	Technical Mechanism & Constraint Handling
1. Local Sensing	Observe adjacent cells	Agent queries 4-directional local sensors; identifies wall blocks and risky zones.
2. Knowledge Map Update	Integrate sensor data into Belief State	Updates internal occupancy grid map G_{known} with real cell costs ($c=1$, $c=3$ for risky, $c=\infty$ for blocked).
3. Local Decision & Path Search	Select best adjacent node	Evaluates $f(a) = c(s, a) + H(a)$. Updates heuristic value of current state $H(s) \leftarrow \max(H(s), f(a))$ to avoid looping.
4. Physical Execution	Move to chosen neighbor cell	Executes actuator step. If new cell contains unexpected dynamic obstacle, triggers re-evaluation.
5. Goal Verification	Check if current position = G	If G is reached, rescue sequence completes; otherwise, repeats from Phase 1.

c) Demonstration of AI Core Concepts

- **Intelligent Agent Framework:** Operates on the PEAS model: Performance Measure = survivor rescue success, cost minimization, zero collisions; Environment = partially observable disaster building; Actuators = tread drive motors, robotic arm; Sensors = local thermal, LiDAR, tactile sensors.
- **Rationality:** The agent exhibits expected utility maximization based on its current percept sequence and internal knowledge base, making optimal choices despite incomplete information.
- **Environment Type Classification:** Partially Observable (cannot see unvisited rooms), Stochastic/Dynamic (aftershocks may create new debris), Sequential (moves alter future options), Continuous time / Discrete grid, Single-Agent operating in a dynamic setting.

d) Justification of Chosen Online Search Approach

Justification Summary:

1. Offline A* or standard Uniform Cost Search fails completely because complete graph topology is unknown beforehand.
2. Online LRTA* / D* Lite dynamically interleaves planning and execution, guaranteeing completeness in finite state spaces.
3. Cost-sensitive heuristic routing handles variable terrain weights ($c=1$ vs $c=3$ for risky zones) seamlessly, ensuring survivor rescue at minimum expected cost.